

Given Prompt:

Generate a C++ program to implement the following requirements

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must use objects instead of functions. Do not include pre-existing heap Modules.

- Test File format:

Command Description

EMAIL <sender category> ,

<subject line> ,

<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

*The test file will begin

with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Gemini Code:

```
#include <iostream>
#include <string>
#include <sstream>
#include <fstream>

// Class representing an Email object
class Email {
private:
    std::string senderCategory;
    std::string subject;
    std::string dateStr;
    int priorityRank; // Lower numerical value = higher priority
    int year;
    int month;
    int day;

    void parseDate(const std::string& date) {
        // Expected format: MM-DD-YYYY
        if (date.length() >= 10) {
            month = std::stoi(date.substr(0, 2));
            day = std::stoi(date.substr(3, 2));
            year = std::stoi(date.substr(6, 4));
        } else {
            year = month = day = 0;
        }
    }

    void assignPriorityRank(const std::string& category) {
        if (category == "Boss") priorityRank = 1;
        else if (category == "Subordinate") priorityRank = 2;
        else if (category == "Peer") priorityRank = 3;
        else if (category == "ImportantPerson") priorityRank = 4;
        else priorityRank = 5; // "OtherPerson"
    }

public:
    Email() : priorityRank(5), year(0), month(0), day(0) {}

    Email(const std::string& category, const std::string& subj, const std::string& date)
        : senderCategory(category), subject(subj), dateStr(date) {
        assignPriorityRank(category);
        parseDate(date);
    }

    std::string getSenderCategory() const { return senderCategory; }
    std::string getSubject() const { return subject; }
    std::string getDateStr() const { return dateStr; }

    // Priority comparison: Returns true if *this has HIGHER priority than 'other'
    bool hasHigherPriorityThan(const Email& other) const {
        if (this->priorityRank != other.priorityRank) {
```

```

        return this->priorityRank < other.priorityRank; // Lower rank number = higher priority
    }
    // If same category, newest email first (larger date)
    if (this->year != other.year) return this->year > other.year;
    if (this->month != other.month) return this->month > other.month;
    return this->day > other.day;
}
};

```

```

// Node for list/pointer-based binary MaxHeap tree
class HeapNode {
public:
    Email data;
    HeapNode* left;
    HeapNode* right;
    HeapNode* parent;

    HeapNode(const Email& email)
        : data(email), left(nullptr), right(nullptr), parent(nullptr) {}
};

```

```

// List-based MaxHeap Priority Queue Implementation
class MaxHeap {
private:
    HeapNode* root;
    int count;

    // Helper method to find node at a specific 1-based level-order index
    HeapNode* getNodeAt(int index) const {
        if (index <= 0 || index > count) return nullptr;
        if (index == 1) return root;

        // Path finding using binary representation of the index
        int path[32];
        int pathLen = 0;
        int temp = index;

        while (temp > 1) {
            path[pathLen++] = temp % 2; // 0 for left, 1 for right
            temp /= 2;
        }

        HeapNode* curr = root;
        for (int i = pathLen - 1; i >= 0; --i) {
            if (path[i] == 0) curr = curr->left;
            else curr = curr->right;
        }
        return curr;
    }

    void bubbleUp(HeapNode* node) {
        while (node->parent != nullptr && node->data.hasHigherPriorityThan(node->parent->data)) {

```

```

        // Swap email data between node and parent
        Email temp = node->data;
        node->data = node->parent->data;
        node->parent->data = temp;

        node = node->parent;
    }
}

void bubbleDown(HeapNode* node) {
    while (node != nullptr) {
        HeapNode* highest = node;

        if (node->left && node->left->data.hasHigherPriorityThan(highest->data)) {
            highest = node->left;
        }
        if (node->right && node->right->data.hasHigherPriorityThan(highest->data)) {
            highest = node->right;
        }

        if (highest != node) {
            Email temp = node->data;
            node->data = highest->data;
            highest->data = temp;

            node = highest;
        } else {
            break;
        }
    }
}

void destroyTree(HeapNode* node) {
    if (node) {
        destroyTree(node->left);
        destroyTree(node->right);
        delete node;
    }
}

public:
    MaxHeap() : root(nullptr), count(0) {}

    ~MaxHeap() {
        destroyTree(root);
    }

    bool isEmpty() const {
        return count == 0;
    }

    int getCount() const {

```

```

    return count;
}

void insert(const Email& email) {
    count++;
    HeapNode* newNode = new HeapNode(email);

    if (count == 1) {
        root = newNode;
        return;
    }

    // Find parent position for newly added node at index 'count'
    HeapNode* parentNode = getNodeAt(count / 2);
    newNode->parent = parentNode;

    if (count % 2 == 0) {
        parentNode->left = newNode;
    } else {
        parentNode->right = newNode;
    }

    bubbleUp(newNode);
}

Email peekMax() const {
    if (!isEmpty()) {
        return root->data;
    }
    return Email();
}

void extractMax() {
    if (isEmpty()) return;

    if (count == 1) {
        delete root;
        root = nullptr;
        count = 0;
        return;
    }

    HeapNode* lastNode = getNodeAt(count);
    root->data = lastNode->data; // Move last element to root

    // Detach last node from parent
    HeapNode* parentNode = lastNode->parent;
    if (parentNode->left == lastNode) {
        parentNode->left = nullptr;
    } else {
        parentNode->right = nullptr;
    }
}

```

```

        delete lastNode;
        count--;

        bubbleDown(root);
    }
};

// CEO Inbox Manager class to process incoming command streams
class EmailManager {
private:
    MaxHeap priorityQueue;

    std::string trim(const std::string& str) {
        size_t first = str.find_first_not_of(" \t\r\n");
        if (first == std::string::npos) return "";
        size_t last = str.find_last_of(" \t\r\n");
        return str.substr(first, (last - first + 1));
    }

public:
    void processCommand(const std::string& line) {
        std::string trimmedLine = trim(line);
        if (trimmedLine.empty()) return;

        if (trimmedLine.rfind("EMAIL ", 0) == 0) {
            std::string data = trimmedLine.substr(6);
            std::stringstream ss(data);
            std::string category, subject, date;

            if (std::getline(ss, category, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, date, ',')) {

                Email email(trim(category), trim(subject), trim(date));
                priorityQueue.insert(email);
            }
        }
        else if (trimmedLine == "COUNT") {
            std::cout << "There are " << priorityQueue.getCount() << " emails to read.\n\n";
        }
        else if (trimmedLine == "NEXT") {
            if (!priorityQueue.isEmpty()) {
                Email top = priorityQueue.peekMax();
                std::cout << "Next email:\n";
                std::cout << "Sender: " << top.getSenderCategory() << "\n";
                std::cout << "Subject: " << top.getSubject() << "\n";
                std::cout << "Date: " << top.getDateStr() << "\n\n";
            }
        }
        else if (trimmedLine == "READ") {
            if (!priorityQueue.isEmpty()) {

```

```

        priorityQueue.extractMax();
    }
}

void processStream(std::istream& in) {
    std::string line;
    while (std::getline(in, line)) {
        processCommand(line);
    }
}

};

int main(int argc, char* argv[]) {
    EmailManager manager;

    if (argc > 1) {
        std::ifstream file(argv[1]);
        if (file.is_open()) {
            manager.processStream(file);
            file.close();
        } else {
            std::cerr << "Error: Could not open file " << argv[1] << std::endl;
            return 1;
        }
    } else {
        // Fallback to reading from standard input
        manager.processStream(std::cin);
    }

    return 0;
}

```

Analysis:

Correctness:

- Execution: in current form does not output anything. The trim logic happens before any commands and if there's no whitespace to trim it returns an empty string.
- Edge cases: ignores mismatched inputs, no fallback for invalid commands. However, there is a way to read in from standard input if there's no passed in file.
- Error handling: gives an error when unable to open the file, no others

Time Complexity:

- Time complexity is actually remarkably good since it's object based. There's no loops or unnecessary variables since all of it is wrapped in specific classes.

Space Complexity

- Space is also pretty good, since the only part of the program that stores data is the EMAIL command, so the vast majority of the program is unaffected by scaling.

Maintanability:

- Comments: only really present to mark new classes, no explanations of command handing or the other large blocks of code.
- Context: names of objects and classes are clear and directly relate to their purpose.

=====

ChatGPT code:

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

```
// -----
```

```
// Email class
```

```
// -----
```

```
class Email {
```

```
private:
```

```
    string sender;
```

```
    string subject;
```

```
    string date;
```

```
    int priority;
```

```
public:
```

```
    Email() {
```

```
        sender = "";
```

```
        subject = "";
```

```
        date = "";
```

```
        priority = 0;
```

```
    }
```

```
    Email(string sender, string subject, string date) {
```

```
        this->sender = sender;
```

```
        this->subject = subject;
```

```
        this->date = date;
```

```

// Higher number = higher priority
if (sender == "Boss")
    priority = 5;
else if (sender == "Subordinate")
    priority = 4;
else if (sender == "Peer")
    priority = 3;
else if (sender == "ImportantPerson")
    priority = 2;
else
    priority = 1;
}

string getSender() const {
    return sender;
}

string getSubject() const {
    return subject;
}

string getDate() const {
    return date;
}

int getPriority() const {
    return priority;
}

// Converts MM-DD-YYYY into an integer that can be compared.
int dateValue() const {
    int month = stoi(date.substr(0, 2));
    int day = stoi(date.substr(3, 2));
    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
}
};

// -----
// MaxHeap class

```

```

// List-based implementation
// -----
class MaxHeap {
private:
    vector<Email> heap;

    int parent(int index) {
        return (index - 1) / 2;
    }

    int leftChild(int index) {
        return 2 * index + 1;
    }

    int rightChild(int index) {
        return 2 * index + 2;
    }

    // Determines whether a has higher priority than b.
    bool higherPriority(const Email& a, const Email& b) {
        // First compare sender category.
        if (a.getPriority() != b.getPriority()) {
            return a.getPriority() > b.getPriority();
        }

        // If the sender categories are equal,
        // the newest email has priority.
        return a.dateValue() > b.dateValue();
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int p = parent(index);

            if (higherPriority(heap[index], heap[p])) {
                swap(heap[index], heap[p]);
                index = p;
            }
            else {
                break;
            }
        }
    }
}

```

```
}
```

```
void heapifyDown(int index) {
```

```
    int size = static_cast<int>(heap.size());
```

```
    while (true) {
```

```
        int left = leftChild(index);
```

```
        int right = rightChild(index);
```

```
        int largest = index;
```

```
        if (left < size &&
```

```
            higherPriority(heap[left], heap[largest])) {
```

```
            largest = left;
```

```
        }
```

```
        if (right < size &&
```

```
            higherPriority(heap[right], heap[largest])) {
```

```
            largest = right;
```

```
        }
```

```
        if (largest != index) {
```

```
            swap(heap[index], heap[largest]);
```

```
            index = largest;
```

```
        }
```

```
        else {
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
public:
```

```
    MaxHeap() {}
```

```
    bool isEmpty() const {
```

```
        return heap.empty();
```

```
    }
```

```
    int size() const {
```

```
        return static_cast<int>(heap.size());
```

```
    }
```

```
    // Insert an email into the MaxHeap.
```

```

void insert(const Email& email) {
    heap.push_back(email);
    heapifyUp(static_cast<int>(heap.size()) - 1);
}

// Return the highest-priority email without removing it.
Email peek() const {
    return heap[0];
}

// Remove the highest-priority email.
void removeMax() {
    if (heap.empty()) {
        return;
    }

    heap[0] = heap.back();
    heap.pop_back();

    if (!heap.empty()) {
        heapifyDown(0);
    }
}
};

```

```

// -----
// CEO Inbox class
// Handles all email operations.
// -----
class CEOInbox {
private:
    MaxHeap emailQueue;

    // Prints the email currently at the front.
    void displayEmail(const Email& email) {
        cout << "Next email:" << endl;
        cout << "Sender: " << email.getSender() << endl;
        cout << "Subject: " << email.getSubject() << endl;
        cout << "Date: " << email.getDate() << endl;
        cout << endl;
    }
}

```

```

public:
    // Add an email to the CEO's inbox.
    void addEmail(const Email& email) {
        emailQueue.insert(email);
    }

    // NEXT displays the highest-priority email,
    // but does NOT remove it.
    void nextEmail() {
        if (emailQueue.isEmpty()) {
            return;
        }

        displayEmail(emailQueue.peek());
    }

    // READ removes the highest-priority email.
    // It does not display anything.
    void readEmail() {
        if (emailQueue.isEmpty()) {
            return;
        }

        emailQueue.removeMax();
    }

    // COUNT displays the number of unread emails.
    void countEmails() {
        cout << "There are " << emailQueue.size()
              << " emails to read." << endl;
        cout << endl;
    }
};

// -----
// Main
// -----

int main(int argc, char* argv[]) {
    CEOInbox inbox;

```

```
// The input file is supplied as a command-line argument.  
if (argc < 2) {  
    cout << "Please provide the test file name." << endl;  
    return 1;  
}
```

```
ifstream inputFile(argv[1]);
```

```
if (!inputFile) {  
    cout << "Unable to open input file." << endl;  
    return 1;  
}
```

```
string line;
```

```
while (getline(inputFile, line)) {  
    if (line.empty()) {  
        continue;  
    }  
}
```

```
// -----  
// EMAIL command  
// Format:  
// EMAIL Sender,Subject,MM-DD-YYYY  
// -----
```

```
if (line.substr(0, 5) == "EMAIL") {  
    string emailData = line.substr(6);
```

```
    string sender;  
    string subject;  
    string date;
```

```
    stringstream ss(emailData);
```

```
    // Read sender up to the first comma.  
    getline(ss, sender, ',');
```

```
    // Read subject up to the second comma.  
    getline(ss, subject, ',');
```

```
    // The remainder is the date.  
    getline(ss, date);
```

```

        Email email(sender, subject, date);

        inbox.addEmail(email);
    }

    // -----
    // NEXT command
    // -----
    else if (line == "NEXT") {
        inbox.nextEmail();
    }

    // -----
    // READ command
    // -----
    else if (line == "READ") {
        inbox.readEmail();
    }

    // -----
    // COUNT command
    // -----
    else if (line == "COUNT") {
        inbox.countEmails();
    }
}

inputFile.close();

return 0;
}

```

Analysis:

Correctness:

- Execution: gives desired output, but doesn't trim whitespace so presentation could be messy
- Edge cases: does nothing on empty files and has no method for mismatched commands
- Error handling: only gives a print out when it is unable to open a file, no other errors

Time Complexity:

- Comparatively terrible. During its heap operations it re-parses the string with dateValue() every time. Not to mention the while loop in main that could have been better implemented as a helper method

Space Complexity

- Using <vector> does allow it to handle memory well, but its usage of variables, loops, and other less optimal tools means it's not very impressive.

Maintanability:

- Comments: to its credit the comments and structure is incredible. Classes and methods are separated and explained very well. Each and every block of code has good comments and their purposes are clear.
- Context: class and variable names are also very good. It was clearly the priority in the LLM's code to focus on readability since it is utilizing very basic methods.

=====

Conclusion:

This time ChatGPT put up a good fight but in terms of effiecentcy and competency Gemini is still the better starting point. Gemini had a few problems with reading lines since it relied on trim() but I still believe it to be a more maintainable program after I patch some holes and finish commenting. ChatGPT was incredibly readable but it is clearly prioritizing that over a more robust program, which given its popularity and general use makes sense.